Validater — ProofGraph Reference Plan

Oct 9, 2026 · @Winners Winners

1. Overview

Validater turns a student's self-reported skills into a verified skill graph. GitHub and LeetCode are the evidence, and a quiz on the student's own code is the proof of understanding.

Problem. Students without networks can't prove what they know. A resume is self-reported, recruiters can't verify it, and students can't see their own gaps. Skilled students get ignored and unskilled ones bluff through.

Solution. A multi-agent system that:

1. Extracts claims from LinkedIn and the resume.
2. Finds evidence for each claim in the student's code, with commit, file and line.
3. Scores every concept Verified, Weak or Unverified using fixed rules.
4. Quizzes the student on their own code before anything becomes Verified.
5. Rescans on every commit, so the score rises only when new learning is proven.

One-line pitch. Proof, not claims: every skill on the profile links to the code that shows it and the quiz that confirms it.

Positioning.

Product	What it does	What it lacks that we have
GitRoll	Scores GitHub profiles against 800+ quality rules for recruiters	Claim-vs-evidence comparison, quiz on own code, student gap loop, college deployment
SkillGraph (open source)	Deterministic capability scoring and gap analysis on synthetic data	Real GitHub ingestion, webhooks, repo-grounded quizzes (all on its unfinished roadmap)
lucidRESUME (open source)	Desktop evidence-backed resume editor	Multi-role platform, live rescans, verification quiz

Our four differentiators:

- Claims compared against evidence, with an Honesty score.
- Verified only after a timed quiz on the student's own code.
- A student-facing loop: see the gap, prove it, watch the score move.
- On-prem deployment for colleges, with cross-batch integrity checks.

2. Core principles

These six rules decide every design choice below. If a feature breaks one, the feature changes.

1. **The LLM never decides the score.** It may normalize claims, propose candidate evidence, explain evidence and write quiz questions. Fixed, versioned rules compute every score.
2. **Every evidence item records its exact source.** student → concept → repository → commit SHA → file and lines → detector → strength → timestamp. Anyone can click through to the line.
3. **A claim alone earns zero.** A resume or LinkedIn line with no evidence stays Unverified.
4. **The quiz is the gate, not a shortcut.** Code evidence alone gives Weak. Code evidence plus a passed quiz gives Verified. A quiz can never verify a claim that has no code behind it.
5. **Isolate data, not infrastructure.** No permanent container per student. Each scan gets a short-lived, locked-down container that is destroyed afterward.
6. **Agents never touch Docker directly.** A runner service owns the Docker socket and enforces CPU, RAM, time, network and filesystem limits.

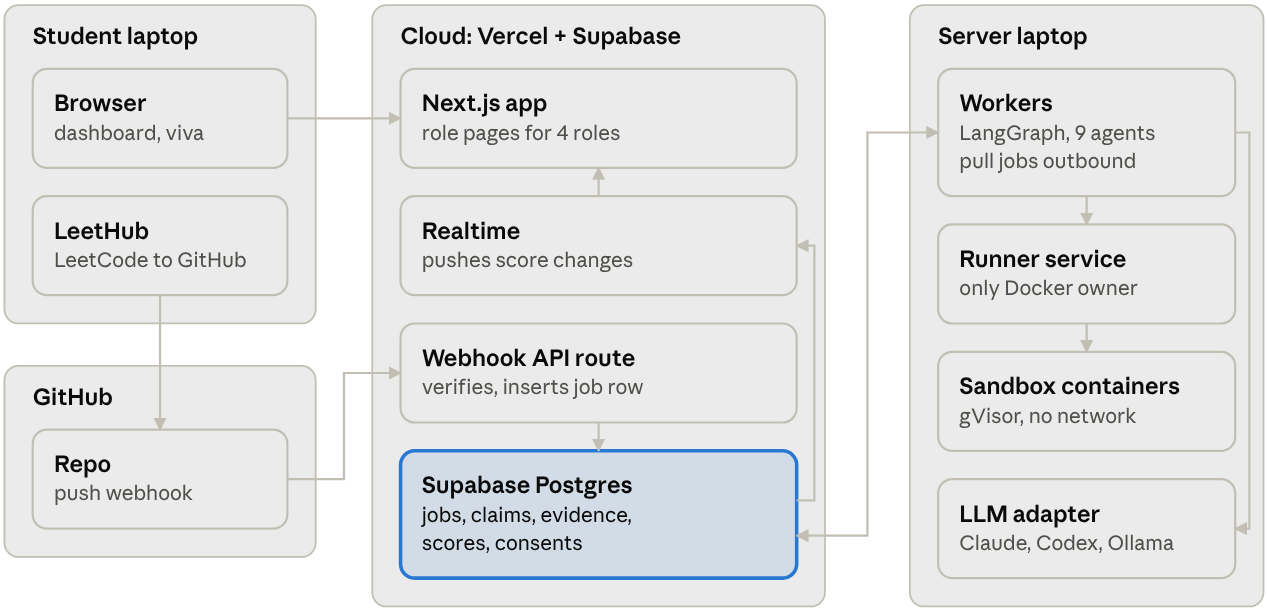
Two honesty rules follow from these:

- We do not run an AI-generated-code detector. Research shows they are unreliable; the quiz does that job.
- Forks, vendored code and repos below an evidence threshold never nominate skills.

3. System topology

Three machines plus GitHub. The student only needs a browser; all analysis runs on the server laptop; the hosted app shows results.

Students and GitHub feed one job queue; the server laptop does all analysis



System topology · 3 nodes plus GitHub

A push to GitHub becomes a job row in Postgres. Workers on the server laptop pull jobs outbound, analyze code in throwaway sandboxes through the runner, and write evidence and scores back. Realtime pushes the new score to the student's open dashboard.

Node	Runs	Role
Student laptop	Browser, LeetHub extension	Connect accounts, take viva, push commits
Cloud (Vercel + Supabase)	Next.js app, webhook route, Postgres, Realtime	All pages, the job queue, all stored data
Server laptop	Workers, runner service, sandbox containers, LLM adapter	Every scan, quiz generation and grading

Because workers pull jobs outbound, the server laptop needs no public address. That is what makes it work on college Wi-Fi.

4. Ingestion

Four sources, none needing an official API we can't get.

Source	How we get it	What we take	Ownership check
GitHub	GitHub App: OAuth plus push webhooks	Repos, commits, file contents, CI config, contributor history	OAuth login proves the account
LinkedIn	Data export ZIP (Skills.csv, Positions.csv) as primary; profile PDF as fallback	Skills, roles, project descriptions, dates	Name must match the GitHub account
Resume	PDF or DOCX upload, parsed with Docling	Skills, projects, claimed technologies	Same as LinkedIn
LeetCode stats	Public GraphQL by username (via alfa-leetcode-api or LeetCode-Query)	Solved counts per tag (fundamental, intermediate, advanced), difficulty split, contest rating	One-time code pasted into the LeetCode bio
LeetCode code	LeetHub v2 extension pushes every accepted solution to a GitHub repo	Solution source for the complexity probe	Arrives through the student's own GitHub

Rules:

- We never ask for LeetCode passwords or session cookies.
- LeetCode is a snapshot plus LeetHub commits, never the centerpiece. Its interfaces are unofficial, so the connector stays replaceable.
- The LinkedIn API is restricted; the export ZIP is the documented fallback for judges.
- Claude structured output turns parsed text into a list of raw claims; it does not score them.

5. Agent pipeline

Nine agents run as one LangGraph graph. Deterministic tools do most of the work; Claude only handles language and ambiguity.

#	Agent	Input	Output	Tools	LLM role
1	Claim Extractor	LinkedIn ZIP/PDF, resume	Normalized claims mapped to ontology concepts	Docling, concept ontology, embedding match	Normalize free text to concepts
2	Code Auditor	Repo snapshot or diff	Evidence items with commit, file, lines, detector, strength	ast-grep rules, tree-sitter, Aider repo map, test runner in sandbox	Classify only ambiguous hits
3	Authorship Checker	Git history	Share of lines the student wrote; fork and vendored-code flags	git-fame, git blame, GitHub fork metadata	None
4	Integrity Agent	Repos across a batch plus known tutorial repos	Similarity clusters and flags	Dolos	None
5	LeetCode Analyzer	Public stats, LeetHub solutions	Topic depth; complexity-probe results	LeetCode GraphQL, Piston	None
6	Verifier	All evidence, claims, quiz results	Per-concept score and status	Scoring rules (section 6)	None
7	Gap Planner	Scores, target role	Ranked gaps with gap type and missions	Role templates from the ontology	Phrase the mission text
8	Examiner	Cited evidence from the latest scan	Pre-generated question set; graded answers	QLC templates from the AST, faster-whisper	Phrase questions, grade against rubric
9	Orchestrator	Webhook or manual job	Incremental rescan of changed files only	LangGraph, Postgres job queue	None

Complexity probe. The LeetCode Analyzer runs each LeetHub solution in Piston at input sizes 10^3, 10^4 and 10^5 and fits the runtime growth. That measures pattern vs. brute force instead of asking an LLM's opinion.

Examiner question types. Built from the student's own code, following the Questions about Learners' Code research:

- Why this structure? ("Why a queue in worker.py instead of a list?")
- What happens if? ("What does `get_user` return when the cache is empty?")
- Trace it. ("Which function runs after the token expires?")

- Change it. ("How would you add rate limiting to this route?")

6. Evidence model and scoring

Each concept gets a 0–100 score from its evidence items, then a status. The formula is adapted from SkillGraph's capability score.

Evidence item fields: student, concept, repository, commit SHA, file path, line range, detector id, detector version, strength tier, authorship share, executed (tests passed), timestamp.

Per-concept score:

```
S(c) = 100 \cdot \left(1 - e^{-\sum_{e \in E(c)} W(e) \cdot \lambda(e) \cdot A(e) \cdot X(e)}\right) \cdot Q(c)
```

Factor	Meaning	Values (starting point, tune later)
W(e)	Detector strength tier	Deep use 1.0, standard use 0.6, trivial use 0.2
λ(e)	Recency decay	Half-life of 730 days from commit date
A(e)	Authorship share	Share of the evidence lines the student wrote, 0–1
X(e)	Execution bonus	1.0 if tests ran green in sandbox, 0.8 otherwise
Q(c)	Quiz multiplier	1.0 passed, 0.7 not attempted, 0.5 failed

Status rules (deterministic):

Status	Rule
Verified	At least one evidence item with A ≥ 0.5, and a passed quiz on that evidence
Weak	Evidence exists but no passed quiz, or authorship below 0.5
Unverified	No evidence at all, whatever the resume says
Flagged	Integrity Agent similarity above threshold; shown to student with appeal option

Two headline scores:

- **Honesty** = claimed concepts with any evidence ÷ all claimed concepts. Rewards claiming only what you can show.
- **Role Readiness** = weighted coverage of the target role's concepts by status (Verified 1.0, Weak 0.5). Stops students from gaming Honesty by claiming less.

Gap types (borrowed from lucidRESUME's career planner):

Gap type	Meaning	Typical fix
Presentation gap	Evidence exists but the skill isn't claimed	Add it to the resume
Weak evidence	Claimed, evidence is thin or unquizzed	Take the quiz or deepen the code
Adjacent skill	Close to a verified skill	Small mission in an existing repo
True gap	No evidence and nothing adjacent	New project or course plus mission

The Gap Planner ranks gaps by how much each would raise Role Readiness per hour of effort.

7. Student features

The student side is a loop: see a gap, prove the skill, watch the score rise. Fifteen features, grouped by job.

Prove

Feature	What it does	Why it matters
Skill Graph	Interactive concept map colored Verified, Weak, Unverified; click a node for its evidence trail	The student sees their whole profile at a glance
Claims and evidence	Each claim with its linked commits, files and lines, or the exact missing evidence	Turns "Unverified" into a concrete to-do

Feature	What it does	Why it matters
Prove-It Missions	Gap Planner turns a gap into a small task in the student's own repo ("add caching to /products"); ast-grep rules and tests check it; then a quiz unlocks	The main way scores rise
Viva Practice	Mock oral exam on the student's own code, voice or text; practice mode doesn't count, ranked mode has a cooldown	Prepares for the quiz gate and real interviews
Integrity and appeals	Explains any flag and lets the student appeal	Fairness and trust

Present

Feature	What it does	Why it matters
Proof Profile	Public shareable page with QR code and embeddable badge; every verified concept links to evidence	Replaces the unverifiable resume line
Resume Checker and Builder	Flags resume lines the student can't prove; writes bullets backed by evidence links	Stops bluffing before applying
Repo Readiness	Suggests README, tests, CI and structure fixes	Makes repos recruiter-ready
Privacy and Consent	Controls who sees what (college, recruiters, public); revocable anytime	Required for recruiter access

Grow

Feature	What it does	Why it matters
Role Fit	Readiness per target role; paste a job description for an instant match	Shows what to learn next

Feature	What it does	Why it matters
Interview Prep	Questions an interviewer would ask about the student's own projects, with feedback	Uses their real work, not generic questions
DSA Insights	LeetCode topic heatmap, complexity-probe results, weak patterns, suggested problem sets	Pattern vs. brute force, measured
Growth Timeline	Score history, log of newly proven skills, streaks	Shows progress, keeps them coming back
Cohort Benchmark	Anonymous percentile within batch and department	Motivation without public ranking
Opportunities	Drives and roles the student qualifies for; one-click apply with Proof Profile	Closes the loop to jobs

8. Pages by role

Five surfaces share one app. Each role sees only its own routes, enforced by row-level security.

Public and shared

Page	Route	Purpose
Landing	`/`	Pitch, how it works, sign-up
Pricing and docs	`/pricing`, `/docs`	Plans for students, colleges, companies
Login	`/login`	GitHub login for students; email or SSO for others
Onboarding wizard	`/onboarding`	Connect GitHub, upload LinkedIn ZIP or resume, LeetCode username, pick target role
Public Proof Profile	`/p/[handle]`	Shareable verified profile
Verify	`/verify/[code]`	Scan a QR code to confirm a profile is genuine

Student (`/student`)

Page	Purpose
Home	Honesty and Readiness scores, next best mission, recent changes
Skill Graph	Interactive concept map
Claims	List of claims; detail page shows evidence or what's missing
Missions	List, detail, submission status
Viva	Practice and ranked attempts, history
Interview Prep	Questions on own projects with feedback
Resume Studio	Checker and builder
Role Fit	Readiness per role, job-description match
DSA Insights	LeetCode heatmap, complexity probe, suggestions
Repo Readiness	Fix list per repo
Growth Timeline	Score history and streaks
Opportunities	Eligible drives and roles, applications
Notifications	Score changes, new missions, recruiter views
Privacy and Consent	Visibility controls
Integrity and Appeals	Flags and appeal form
Settings	Connected accounts, LeetCode ownership check

College admin (`/college`) — placement officer or faculty

Page	Purpose
Batch dashboard	Readiness by department, year, section
Student directory	Search and drill into individual profiles
Curriculum gaps	Concepts the batch lacks, workshop suggestions

Page	Purpose
Assign missions	Push missions to a class or section
Integrity clusters	Groups of students with suspiciously similar repos
Placement drives	Company requirements in, auto-generated eligible list out
Company partnerships	Drive requests and invitations
Reports	Placement readiness and accreditation exports
Roster	CSV import and invites
Infrastructure (on-prem only)	Worker usage, queue, scan times

Company (`/company`) — recruiter

Page	Purpose
Talent search	Filter by verified concepts, college, batch, readiness
Role profiles	Paste a job description; it becomes required concepts
Candidate view	Evidence, quiz transcripts, integrity status (consented data only)
Shortlist pipeline	Saved, contacted, interviewing, offered
Custom challenges	Send a repo-specific quiz or mission to a candidate
Campus connect	Request drives with partner colleges
Analytics and billing	Usage, hires, invoices

Platform admin (`/admin`) — our team

Page	Purpose
Tenants	Colleges, companies, plans

Page	Purpose
Ontology and rules	Concepts, ast-grep rules, role templates, with versions
Cluster monitor	Workers, queue depth, tokens, LLM provider failover
Integrity review	Flag queue and appeals
Quiz quality	Flagged questions, grading disputes
Audit and data requests	Audit log, consent and deletion requests
Feature flags and billing	Rollouts and plan management

9. Technical setup

Two languages only: TypeScript for the web app, Python for everything behind it.

Layer	Choice	Notes
Frontend	Next.js (App Router) on Vercel	Route groups `/student`, `/college`, `/company`, `/admin`; shadcn/ui; Cytoscape.js or React Flow for the Skill Graph; Recharts for charts
Auth	Supabase Auth	GitHub OAuth for students; email or SSO for other roles
Access control	Postgres row-level security	Keyed on organization and role; consent rows gate every recruiter read
Database	Supabase Postgres	Holds the skill graph as tables; pgvector added only when claim matching needs it
Job queue	Postgres jobs table	Workers claim jobs with `SELECT … FOR UPDATE SKIP LOCKED`; no Redis needed

Layer	Choice	Notes
Webhooks	Vercel API route	GitHub push → verify signature → insert job row. Server laptop pulls jobs outbound, so no tunnel on college Wi-Fi
Orchestration	LangGraph (Python)	One graph, nine agents, incremental rescans
Runner service	Small Python service that alone talks to Docker	Enforces CPU, RAM, time, no network, read-only filesystem; agents call the runner, never Docker
Sandbox	Docker with gVisor runtime	One short-lived container per scan, destroyed after
Code execution	Piston	LeetCode complexity probe
Static analysis	ast-grep, tree-sitter, git-fame, Dolos	Deterministic detectors with file and line output
LLM adapter	One interface, three backends	Demo: `claude -p` and `codex exec` with failover. Production: Anthropic API. On-prem: Ollama local model
Speech	faster-whisper on the server	Viva voice answers
Live updates	Supabase Realtime	Score changes push to open dashboards, no refresh

Deployment modes

Mode	Where things run	Who it's for
Hackathon demo	Vercel + Supabase cloud; one server laptop runs workers and runner	Judges
SaaS	Vercel + Supabase; cloud worker machines	Students and companies
College on-prem	Docker Compose with self-hosted Supabase and Ollama on campus servers	Colleges that need data to stay on campus

Security

- Repo code stays in the sandbox; only selected snippets go to the LLM.
- OAuth tokens are encrypted at rest.
- Every recruiter view of a student is logged.
- Personal Claude and Codex subscriptions are for development and the supervised demo only. Their terms are for individual use, so production uses API keys.

10. Data model

Sixteen core tables in Postgres. The skill graph is just concepts, edges and evidence rows; no graph database needed.

Table	Key columns	Purpose
orgs	id, type (college, company, platform), name, plan	Tenants
users	id, org_id, role, github_login, handle	Everyone who logs in
accounts	user_id, provider, external_id, token (encrypted), verified_at	GitHub, LeetCode links
concepts	id, slug, name, category, ontology_version	The ~120-concept ontology
concept_edges	from_concept, to_concept, kind (prerequisite, adjacent)	Gap types and mission paths

Table	Key columns	Purpose
detector_rules	id, concept_id, engine (ast-grep, test, ci), rule, tier, version	Versioned evidence rules
role_templates	id, name, concept_weights (json)	Target roles
claims	id, user_id, concept_id, source (linkedin, resume), raw_text	What the student says
evidence	id, user_id, concept_id, repo, commit_sha, file, line_start, line_end, detector_id, tier, authorship, executed, created_at	What the code shows
scores	user_id, concept_id, score, status, rules_version, computed_at	Current status per concept, plus history
quizzes	id, user_id, evidence_ids, questions (json), answers (json), result, duration	Viva and gate quizzes
missions	id, user_id, concept_id, spec, acceptance_rules, status	Prove-It Missions
integrity_flags	id, user_id, repo, similar_to, similarity, status	Dolos results and appeals
consents	user_id, audience (college, company, public), scope, granted_at, revoked_at	Who can see what
jobs	id, kind, payload, status, attempts, locked_by, timings	Scan queue and worker metrics
audit_log	actor, action, target, at	Every recruiter view and admin action

11. Scaling and metrics

We scale by adding worker machines, never by giving students containers. The unit of capacity is one worker (1 vCPU, 2 GB RAM) plus a token budget.

What we measure live (shown on the admin and college infrastructure pages):

- Students scanned per worker-hour
- Median and p95 scan duration (full scan and incremental rescan)

- Maximum concurrent scans per worker
- Queue depth and wait time
- LLM tokens and cost per student
- Share of score changes backed by new evidence

Growth path:

1. Demo: one server laptop, 2–4 workers.
2. Single college: one or two campus servers, Docker Compose.
3. Many colleges: more worker machines pulling from the same jobs table.

The pitch uses measured numbers from step 1 to project steps 2 and 3. Kubernetes, Firecracker and Temporal stay out until real load needs them.

12. Demo scope and script

The demo proves the whole loop on one student and one repo. Everything else is shown as 1–2 mocked screens.

Build for the demo:

- [ ] LinkedIn ZIP import, with PDF fallback
- [ ] One GitHub repo connected through the GitHub App
- [ ] About 10 concepts with deterministic detectors (auth, REST APIs, DB design, async, caching, tests, CI, error handling, queues, config)
- [ ] Evidence items citing commit, file and lines
- [ ] Honesty and Readiness scores; one gap explained with its gap type
- [ ] Webhook rescan of changed files only
- [ ] Three-question timed viva generated from cited code
- [ ] Live score update on the dashboard
- [ ] Worker metrics panel
- [ ] Optional: LeetHub-synced solve moving the DSA score

Mock only (screens, no backend): missions, college dashboard, company search, integrity clusters.

3-minute script:

Time	Step	What judges see
0:00	Connect	GitHub login, LinkedIn ZIP upload, LeetCode username
0:30	Scores	Claim list, Honesty and Readiness scores, Skill Graph
0:50	Gap	Click an Unverified claim; exact missing evidence and gap type
1:10	Commit	Push a pre-staged caching commit; only changed files rescan; Caching flips to Weak
1:40	Viva	Three timed questions on that diff; pass; Caching flips to Verified
2:20	LeetCode (optional)	Solve a problem; LeetHub pushes it; DSA score moves
2:40	Scale	Worker metrics: scans per worker-hour, p95 scan time, cost per student
2:55	Close	College and company screens as roadmap

Backups: pre-computed baseline scan, cached quiz questions, and a screen recording of the full run.

13. Business model

Students get the score free; colleges and companies pay.

Customer	Free	Paid
Students	Score, Skill Graph, Proof Profile, basic gaps	Full roadmap, unlimited missions and viva practice, re-verification
Colleges	Pilot for one batch	Batch readiness dashboard, integrity clusters, curriculum insights, reports, on-prem license

Customer	Free	Paid
Companies	Limited search	Search by verified concepts, candidate evidence views, custom challenges, campus drives

Defensibility: the evidence-rule library and the historical verification graph. Both improve with every student and every rescan, and nobody else has them.

14. Risks and mitigations

Risk	Mitigation
LinkedIn API is restricted	LinkedIn data export ZIP, with PDF fallback
LeetCode has no official API or OAuth	Public stats by username, bio-code ownership check, LeetHub for code; connector kept replaceable; never ask for passwords or cookies
Copied or tutorial repos	Authorship share from git blame, fork and vendored-code filtering, Dolos similarity checks
AI-written code	Quiz is the gate; detectors are not used because they are unreliable
Quiz cheating	Questions tied to the student's own diff, timed, ranked attempts with cooldown
LLM inconsistency	LLM never sets scores; fixed versioned rules do
Running untrusted code	Runner service, gVisor, no network, CPU, RAM and time limits, container destroyed after each scan
Subscription usage limits during demo	Claude and Codex failover, cached questions, pre-computed baseline, recording
Production model access	API keys in production; Ollama for on-prem
Student privacy	Consent per audience, revocable; audit log of every recruiter view; on-prem option
Similar projects exist (GitRoll, SkillGraph)	Lead with what they lack: claims vs. evidence, quiz gate, live rescans, college on-prem

15. Open-source components

We borrow tools and ideas; none of these projects becomes our foundation.

Project	License	What we use	How
LangGraph	MIT	Agent orchestration	Direct dependency

Project	License	What we use	How
Docling	MIT	Resume and LinkedIn PDF parsing	Direct dependency
ast-grep + tree-sitter	MIT	Concept detectors with file and line output	Direct dependency; our YAML rules are the IP
Aider repo map	Apache 2.0	Picking important code for the LLM	Idea and reference implementation
git-fame	MIT	Authorship share from git blame	Direct tool
Dolos	MIT	Similarity and plagiarism detection	Direct tool
Piston	MIT	Sandboxed code execution for the complexity probe	Self-hosted service
gVisor	Apache 2.0	Container isolation	Docker runtime
alfa-leetcode-api / LeetCode-Query	Check before use	Public LeetCode stats	Reference or dependency, replaceable
LeetHub v2	Check before use	Syncs accepted solutions to GitHub	Students install the extension
Octokit	MIT	GitHub API and webhooks	Direct dependency (or PyGithub in Python)
faster-whisper	MIT	Viva speech-to-text	Direct dependency
SkillGraph	MIT	Scoring formula and dashboard layout	Reference only (synthetic data, early project)
lucidRESUME	Unlicense	Evidence ledger design, four gap types, fork threshold rule, LinkedIn ZIP import	Ideas only (.NET desktop app)

Project	License	What we use	How
GrillKit	Apache 2.0	Viva flow: timers, 1–5 scoring, up to 2 follow-ups, Whisper	Pattern reference
developer-roadmap	Check before use	Seed list for the concept ontology	Reference only

Research to cite in the pitch: Lehtinen et al., Automated Questions about Learners' Own Code Help to Detect Fragile Knowledge (ITiCSE 2023) — working code is not the same as understanding.

16. Build order and roadmap

Build the evidence engine first; the pages are thin until it works.

Hackathon build (in order):

1. **Ontology and rules.** ~10 demo concepts, their ast-grep rules, tiers and two role templates. This is the core IP.
2. **Ingestion.** GitHub App, webhook route, jobs table, LinkedIn ZIP and PDF parsing.
3. **Runner and sandbox.** Runner service, gVisor containers, limits.
4. **Auditor and Verifier.** Detectors, authorship, scoring rules, statuses, Honesty and Readiness.
5. **Examiner.** Question generation from cited evidence, timed viva, grading.
6. **Student dashboard.** Home, Skill Graph, Claims, Viva, live updates.
7. **Metrics and polish.** Worker metrics panel, pre-computed baseline, recording.
8. **Optional.** LeetCode stats, LeetHub sync, complexity probe.

After the hackathon:

1. Prove-It Missions and Gap Planner with all four gap types.
2. Proof Profile, Resume Studio, Role Fit, Interview Prep.
3. College admin portal and Integrity Agent (Dolos).
4. Company portal, consent flows, custom challenges.
5. On-prem package with Ollama and self-hosted Supabase.
6. Expand the ontology from ~10 to ~120 concepts and add languages.

Open questions:

- Which two target roles to template first (suggested: Backend SDE, Full-stack)?
- Quiz pass threshold: 2 of 3 or 3 of 3?
- Which language does the demo repo use? Detectors are written per language.